

RAMP: A Multi-Stage Safety Classifier for Accumulating Prompt, Internal, Output, and Session Risk Signals

Ratnaditya J.

RAMP Reference Implementation

<https://github.com/Ratnaditya-J/RAMP>

Research draft

Abstract

Safety classification for large language model (LLM) and agent systems is commonly framed as a single prompt- or response-level decision. Real deployments, however, expose staged evidence: the user prompt, model-internal representations, generated output, multi-turn session history, and proposed tool actions. We present RAMP, a reference implementation and research design for a multi-stage safety classifier that accumulates such evidence into a versioned risk state. The current RAMP reference implementation implements prompt classification, GPT-OSS input-embedding proximity, GPT-OSS activation probes, generated-output classification, session classification, and tool/action gating. On the current human-reviewed prompt set, deterministic split-stability calibration selects prompt+activation as the primary runtime policy: prompt weight 0.25, activation weight 0.75, input-embedding weight 0.00, and threshold 0.53. This policy reaches mean AUROC 0.9939, mean recall 0.9626, and mean false positive rate 0.0397 across 30 deterministic reviewed-label splits. Input embeddings, output classification, and full-transcript session scoring are useful as taxonomy, audit, post-generation, or escalation evidence, but they do not yet justify positive runtime blocking weight. The main finding is therefore not that every signal improves runtime blocking, but that a staged reference design can make signal value, negative results, and provenance explicit enough for reproducible safety research.

1 Introduction

Safety filters for LLM systems are often evaluated as one-shot classifiers: given a prompt or model response, return safe or unsafe. This framing is useful but incomplete. Interactive and agentic systems produce additional evidence over time: multiple prompts in a session, internal model states, generated responses, tool calls, and action arguments. A prompt may look ambiguous in isolation but become high risk after session context accumulates; a generated answer may be safer or less safe than the original prompt suggested; an internal activation pattern may separate hard benign near-neighbors from harmful requests. These observations motivate a multi-stage classifier rather than a single-stage guard.

RAMP tests this design hypothesis. It is not proposed as a replacement for existing safety classifiers such as Qwen3Guard, Llama Guard, WildGuard, or commercial moderation APIs. Instead, RAMP treats those classifiers as one stage in a broader evidence pipeline. Each stage emits a versioned score, label, confidence, metadata, and provenance record. A fusion policy then combines the available evidence and records which signals were decisive, missing, ignored, or reserved for audit.

This paper reports the current research result. The RAMP reference implementation is feature-complete, but not production-grade or paper-final. Its strongest current evidence supports a prompt+activation runtime core. Other stages are implemented and measured, but currently serve audit, taxonomy, post-generation, or escalation roles. We include these limited and negative results because they are central to the contribution:

RAMP provides a disciplined way to test whether signals earn runtime weight instead of assuming every added stage should improve decisions.

The contributions are:

- A versioned multi-stage safety-classifier reference design spanning prompt, embedding, activation, output, session, and tool/action evidence.
- A GPT-OSS-based internal-signal path that distinguishes input embeddings from hidden-state activations and evaluates them separately.
- A reviewed-label calibration protocol that freezes weights by split-stability rather than manual preference.
- A consolidated evidence package showing which signals currently earn runtime weight and which remain audit/escalation signals.

2 Related Work

Guard classifiers. Recent open guard models provide prompt- and response-level safety classification. Llama Guard frames input-output safeguarding for human-AI conversations (Meta AI, 2023). WildGuard contributes an open moderation model and training mixture for prompt/response safety (Han et al., 2024). Qwen3Guard provides generative safety classification models used in our prompt and output scoring experiments (Qwen Team, 2025). These systems motivate RAMP’s front-door and post-generation classifier stages.

Safety benchmarks and datasets. RAMP uses benchmark-derived rows for representation extraction and targeted review. HarmBench provides a standardized harmful-behavior benchmark for automated red teaming (Mazeika et al., 2024). BeaverTails supplies safety-alignment data for helpfulness and harmlessness analysis (Ji et al., 2023). R-Judge, MHJ, and SafeDialBench motivate multi-turn/session evaluation (Yuan et al., 2024; Scale AI, 2024; Zhang et al., 2025). Because these sources have different label semantics and class balance, RAMP treats them as distinct evidence sources rather than a single homogeneous benchmark.

Internal representations and open-weight targets. RAMP separates input embeddings from activations. Input embeddings are the token-embedding vectors produced before transformer-layer computation; activations are hidden states from later layers. For current extraction, both are taken from the open-weight GPT-OSS path (OpenAI, 2025), which keeps the representation source tied to an instrumentable target model.

Policy and constitutional classifiers. Constitutional or policy-driven classifiers demonstrate that safety decisions can be encoded as explicit policies with calibrated model judgments (Anthropic, 2025). RAMP is compatible with this direction but focuses on accumulating heterogeneous evidence and preserving stage-level provenance.

3 RAMP Method

3.1 Staged Evidence

Figure 1 summarizes the RAMP pipeline.

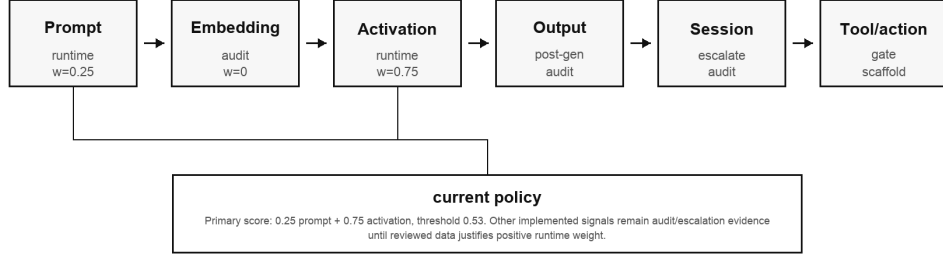


Figure 1: RAMP accumulates staged evidence. The frozen runtime core uses prompt and activation scores. Embedding, output, and session signals are retained as audit/escalation evidence until larger reviewed datasets justify positive runtime weight.

The stages are:

1. **Prompt classifier:** Qwen3Guard scores the incoming prompt.
2. **Input-embedding proximity:** GPT-OSS input embeddings are compared with harmful and benign near-neighbor centroids.
3. **Activation probe:** a linear probe scores GPT-OSS hidden-state activations.
4. **Output classifier:** generated responses are scored after generation.
5. **Session classifier:** compact session evidence and full transcript evidence are scored for multi-turn risk.
6. **Tool/action gate:** proposed tool calls and action arguments are checked by a deterministic reference gate.

3.2 Fusion Policy

Let s_p be the prompt risk score, s_a the activation probability, and s_e the embedding prior. The frozen input-side runtime score is

$$s_{\text{RAMP}} = 0.25s_p + 0.75s_a + 0.00s_e. \quad (1)$$

The selected operating threshold is 0.53. This score is wrapped by severity-aware floors: if the prompt classifier marks a high-severity unsafe category, the final risk can be raised to a configured minimum. The floor is a policy guardrail around calibrated fusion, not another learned weight.

The multi-stage policy artifact also defines non-primary roles. Output scoring is a post-generation audit signal. Session scoring is a calibrated escalation/audit signal. Tool/action gating is a deterministic reference gate pending benchmark validation.

Signal	Final construction	Current role	Selection evidence
Prompt classifier	Qwen3Guard prompt-risk score	Positive runtime signal	Reviewed labels showed useful ranking after noisy benchmark labels were separated.
Embedding proximity	GPT-OSS input embeddings against harmful and benign centroids	Taxonomy/audit signal, runtime weight 0	Useful semantic boundary signal, but no lift over prompt+activation.
Activation probe	Linear probe over GPT-OSS layer-19 hidden states	Positive runtime signal	Layer comparison and reviewed calibration made this the strongest internal signal.
Output classifier	Qwen3Guard score over generated response	Post-generation audit signal	Output-inclusive fusion did not improve the best input-side result.
Session classifier	Compact evidence and full transcript evidence	Escalation/audit signal	Full transcript catches misses; compact state is lossy and max fusion raises FPR.
Tool/action gate	Deterministic check over proposed tool arguments	Reference gate	Implemented pattern; dedicated benchmark still pending.

Table 1: Signal construction and disposition in the current RAMP reference implementation. This table separates implementation status from runtime weight: a signal can be implemented and useful without earning positive blocking weight.

4 Data and Artifacts

Table 2 lists the major datasets and artifacts used in the current study. Large generated artifacts are intentionally not committed to Git; the repository records their paths, summaries, and provenance in the artifact registry.

Artifact or dataset	Purpose	Size
Benchmark-derived corpus	embedding and activation extraction	27,718 rows
Reviewed prompt set	prompt/internal fusion calibration	448 binary reviewed rows
Output eval set	prompt/response output-classifier calibration	134 rows
R-Judge	labeled multi-turn session evaluation	555 sessions
MHJ	unsafe-only multi-turn jailbreak stress test	496 sessions
SafeDialBench	session mining and future review	4,053 sessions

Table 2: RAMP data sources. The corpora are used for different experimental questions and are not treated as a single merged benchmark.

5 Experiments

The experiments are organized by signal family and by the decision each signal must earn. We do not assume that an implemented signal should automatically receive positive runtime weight. Instead, a signal

is assigned one of three roles: primary runtime evidence, audit/taxonomy evidence, or escalation/post-generation evidence.

5.1 Prompt Classifier

The prompt classifier is the front-door signal. We use Qwen3Guard as the open safety classifier rather than training a new text classifier from scratch. Early benchmark-derived labels made the prompt classifier look weak because span-derived labels were not clean prompt-level moderation truth. We therefore built human-reviewed disagreement batches and used those reviewed rows for calibration. After this correction, prompt risk became the stable base signal in the fusion policy.

5.2 Input-Embedding Proximity

The embedding signal uses GPT-OSS input embeddings, not a separate embedding model. For each span, token IDs are mapped through the target model’s input-embedding table, mean-pooled over the prompt, and normalized. We then compare the resulting vector with harmful centroids and benign near-neighbor centroids. Several variants were evaluated, including raw cosine, centered cosine, and domain-conditioned benign contrast. The final interpretation is conservative: embeddings separate broad semantic neighborhoods and are useful for taxonomy review, but their overlap with hard benign near-neighbors prevents them from earning positive runtime weight in the current policy.

5.3 Activation Probe

The activation signal uses GPT-OSS hidden states. We extracted mid-layer, late-layer, and final hidden-state representations and trained linear probes. The initial layer comparison selected layer 19 under a low-FPR rule. We then retrained and recalibrated against the expanded reviewed-label set, including hard cases where activations previously failed. Layer-19 activation probability remained the strongest internal signal and earned the largest runtime weight in the frozen policy.

5.4 Input-Side Fusion

The input-side fusion experiment combines the three pre-generation signals: prompt classifier, embedding proximity, and activation probe. We compare prompt-only, prompt+embedding, prompt+activation, and prompt+embedding+activation on human-reviewed labels. We select weights by deterministic split stability rather than hand tuning. The decisive comparison for the frozen policy is prompt+activation versus prompt+embedding+activation.

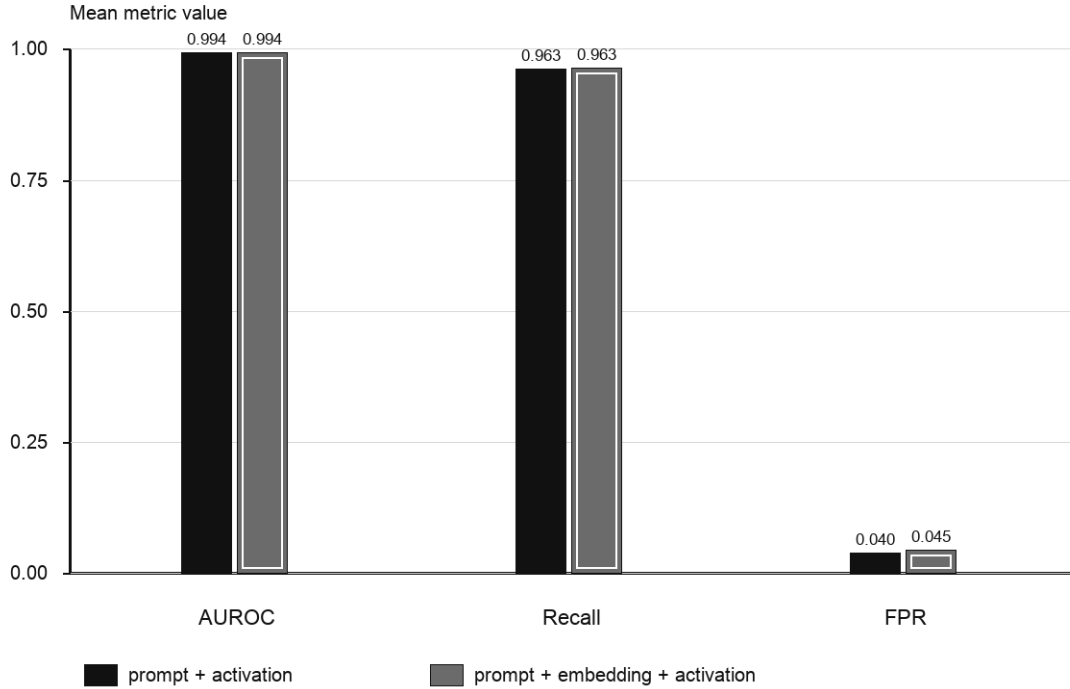


Figure 2: Input-side reviewed split stability. Adding embedding to prompt+activation ties AUROC but increases mean false-positive rate, so embedding is assigned zero runtime weight in the current policy.

Condition	AUROC	Recall	FPR	FP	FN
Prompt+activation	0.9939	0.9626	0.0397	3.73	4.87
Prompt+embedding+activation	0.9939	0.9633	0.0450	4.23	4.77

Table 3: Input-side split-stability means over 30 deterministic reviewed-label splits.

5.5 Output Classifier

The output classifier tests whether generated text adds evidence that prompt-side features miss. We built a 134-row prompt/response set from reviewed prompt examples, generated responses separately, then scored the generated response text with the same Qwen3Guard interface. This intentionally keeps prompt risk and output risk separate: the output classifier is evaluated on model behavior, not on the original prompt text. Table 4 and Figure 3 show that output-inclusive fusion does not improve the strongest input-side result on the current response set. We therefore retain output scoring as post-generation audit and compliance evidence, not as a positive runtime blocking weight.

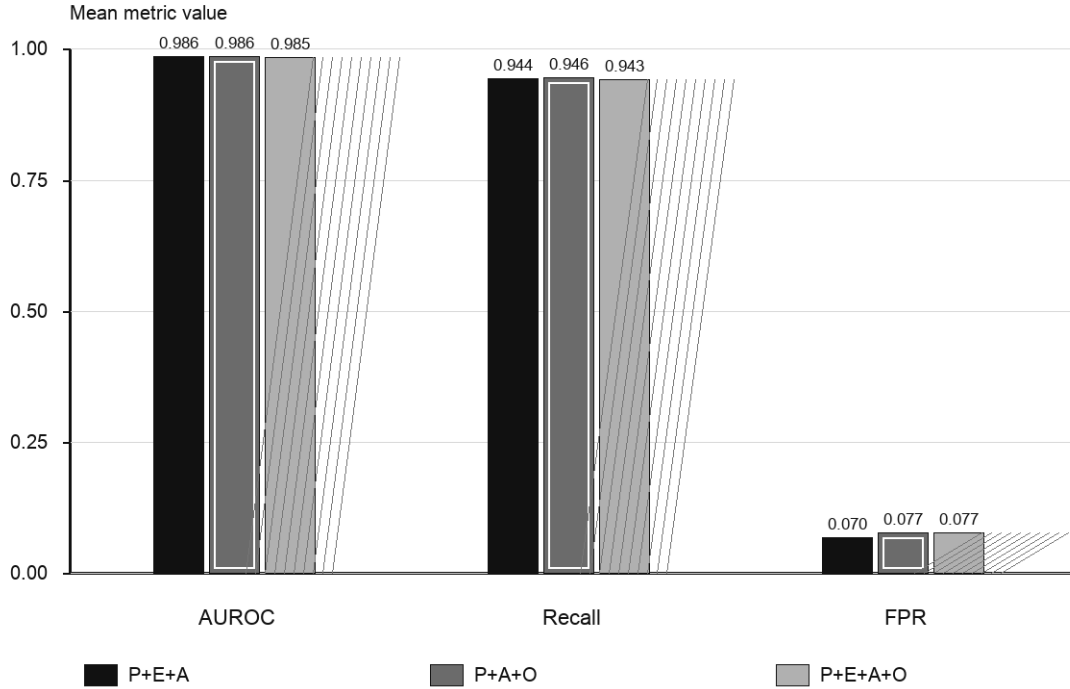


Figure 3: Output-classifier ablation. Output scoring is implemented and useful for audit, but the current prompt/response set does not justify positive runtime fusion weight.

Condition	AUROC	Recall	FPR
Prompt+embedding+activation	0.9864	0.9443	0.0697
Prompt+activation+output	0.9858	0.9457	0.0773
Prompt+embedding+activation+output	0.9851	0.9429	0.0773

Table 4: Output-inclusive calibration over 20 deterministic splits.

5.6 Session Classifier

The session experiments address a separate question: can multi-turn evidence catch risk missed by single-turn scoring? We first tried mathematical accumulation over isolated turn scores; that negative result motivated a real session classifier. We then built two representations. The compact representation contains a safety-specific state: salient turns, domains, severity, trend, composition cues, and evasion/operational-detail indicators. The full-transcript representation is an oracle-like baseline that sends the role-aware transcript to the classifier. We compare both against single-turn maximum score and against naive max fusion. R-Judge provides labeled session metrics; MHJ is unsafe-only and is therefore used only for unsafe-recall stress.

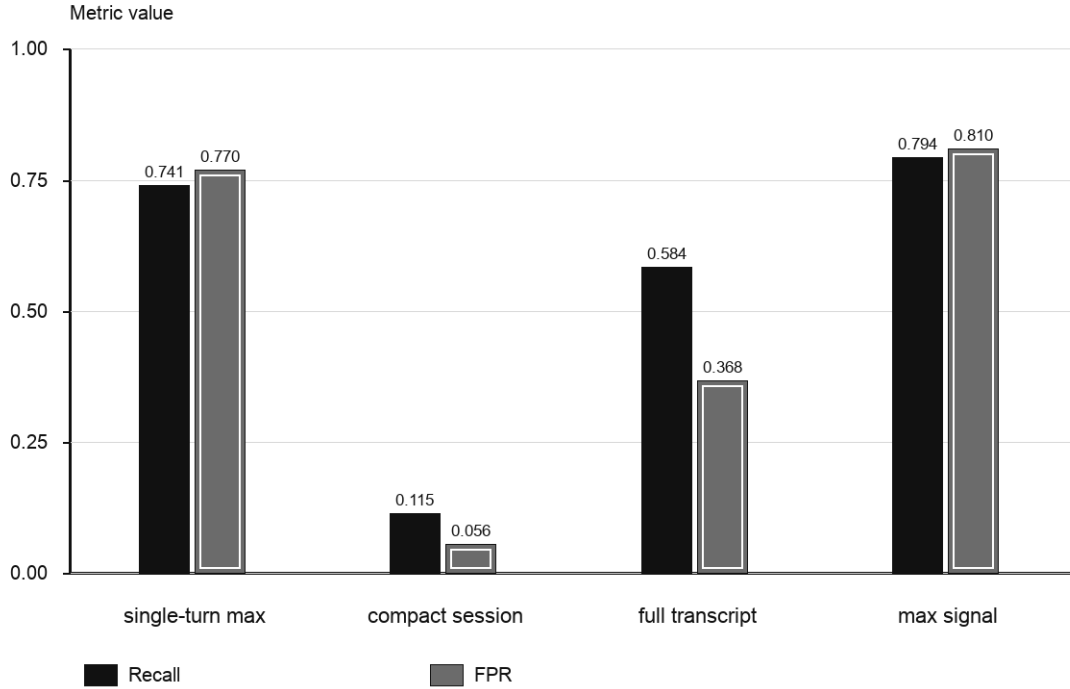


Figure 4: R-Judge session ablation. Full transcript catches single-turn misses, but naive max fusion raises false positives. Compact state is efficient but too lossy in the current implementation.

Condition	AUROC	Recall	FPR	FNs caught
Single-turn max	0.5471	0.7413	0.7695	0
Compact session	0.5299	0.1154	0.0558	1
Full transcript	0.6105	0.5839	0.3680	14
Max session signal	0.5570	0.7937	0.8104	15

Table 5: R-Judge session comparison at threshold 0.55. “FNs caught” counts unsafe sessions missed by single-turn max but caught by the session condition.

Condition	Recall	Single-turn FNs caught
Single-turn max	0.7984	0
Compact session	0.4133	4
Full transcript	0.6956	24
Max session signal	0.8508	26

Table 6: MHJ unsafe-only stress comparison at threshold 0.55. No AUROC or FPR is reported because the current MHJ slice has no safe sessions.

5.7 Tool/Action Gate

The tool/action stage is included because agent systems can be safe in text while risky in proposed actions or arguments. In the current implementation this stage is a deterministic reference gate over tool name, action type, argument sensitivity, and permission context. It is not included in the quantitative tables because we have not yet built a dedicated benchmark of benign and harmful tool/action proposals. The correct current role is therefore design coverage rather than empirical claim.

6 Discussion

What earns runtime weight? The reviewed-label evidence supports prompt+activation as the runtime core. Activation receives high weight not because later-layer signals are assumed superior, but because it repeatedly earned that role under reviewed-label calibration.

What does not earn runtime weight yet? Embedding proximity is useful for taxonomy review and semantic boundary analysis, but it did not improve the frozen runtime tradeoff. Output scoring is useful for post-generation audit and compliance measurement, but the current response set does not justify positive weight. Full transcript session scoring catches missed unsafe sessions, but naive OR/max fusion is false-positive heavy. Compact session evidence is too lossy in its current form.

Negative results as contribution. The project would be weaker if it simply added every signal to a final score. RAMP’s contribution is the ability to evaluate each signal’s role under a declared protocol. Some implemented features are therefore deliberately assigned zero runtime weight.

7 Limitations

The reviewed prompt set is targeted toward hard disagreements and is not a broad deployment distribution. Some session datasets are unsafe-only or unlabeled, limiting AUROC and FPR claims. The activation probe is trained and evaluated on extracted GPT-OSS representations, not on a production-serving stack. Output and session experiments are small-scale and need more human-reviewed coverage. Tool/action gating is implemented as a reference design but lacks benchmark validation. Finally, all thresholds and weights are research artifacts, not production safety guarantees.

8 Reproducibility

The repository records the frozen policies, report generator, and artifact registry. The consolidated report is generated by:

```
python scripts/build_v0_consolidated_report.py \
  --output-md docs/reports/ramp_v0_consolidated_research_report.md
```

The same script can also emit an ignored JSON summary with `--output-json`. The paper figures are generated by:

```
python scripts/build_paper_assets.py
```

The paper source, generated figures, and report artifacts are versioned in the project repository.

9 Conclusion

RAMP is a feature-complete research reference implementation for multi-stage safety classification. Its current paper-grade lesson is careful and bounded: prompt+activation is the best-supported runtime core; embedding, output, and session signals add audit or escalation value but require stronger reviewed evidence before they become positive blocking weights. This result supports RAMP as a useful framework for studying cumulative safety evidence, while preserving clear limitations around data scale, calibration, and deployment readiness.

References

- Anthropic. Constitutional classifiers: Defending against universal jailbreaks across thousands of hours of red teaming, 2025. URL <https://arxiv.org/abs/2501.18837>.
- Seungju Han et al. Wildguard: Open one-stop moderation tools for safety risks, jailbreaks, and refusals of llms, 2024. URL <https://arxiv.org/abs/2406.18495>.
- Jiaming Ji et al. Beavertails: Towards improved safety alignment of llm via a human-preference dataset, 2023. URL <https://arxiv.org/abs/2307.04657>.
- Mantas Mazeika et al. Harmbench: A standardized evaluation framework for automated red teaming and robust refusal, 2024. URL <https://arxiv.org/abs/2402.04249>.
- Meta AI. Llama guard: Llm-based input-output safeguard for human-ai conversations, 2023. URL <https://arxiv.org/abs/2312.06674>.
- OpenAI. gpt-oss model card, 2025. URL <https://arxiv.org/abs/2508.10925>.
- Qwen Team. Qwen3guard technical report, 2025. URL <https://arxiv.org/abs/2510.14276>.
- Scale AI. Multi-turn human jailbreaks on large language models, 2024. URL <https://arxiv.org/abs/2408.15221>.
- Tongxin Yuan et al. R-judge: Benchmarking safety risk awareness for llm agents, 2024. URL <https://arxiv.org/abs/2401.10019>.
- Zhexin Zhang et al. Safedialbench: A fine-grained benchmark for safety evaluation in multi-turn dialogues, 2025. URL <https://arxiv.org/abs/2502.11090>.